LOAN APPROVAL SYSTEM

Vityarthi Platform

NAME: VISHAL SONI

REGISTRATION NO: 25BAI10695

BRANCH: CSE(AI & ML)

YEAR:2025-2029

COURSE CODE: CSA2001

COURSE: Fundamentals of AI & ML

DATE: 30th March,2026

INTRODUCTION

This report describes a Machine Learning application developed to predict loan eligibility. The system automates the decision-making process for financial institutions by analyzing applicant profiles. This project introduces the Intelligent Loan Approval Prediction System, a robust computational framework designed to automate this evaluation using advanced Machine Learning techniques.

Project Objectives

The main goals of this project are:

1. Understand what factors influence loan approval decisions
2. Clean and prepare real-world loan application data
3. Build and compare multiple ML classification models
4. Select the best-performing model and evaluate it rigorously
5. Demonstrate that the model can predict outcomes for new, unseen applicants
6. Save the trained model so it can be deployed in a real system

Dataset Overview

The dataset used in this project is a CSV file named loan_approval_dataset.csv, loaded directly into Google Colab.

Property	Details
Total Records	4,269 loan applications
Total Features	13 columns (including the target variable)
Target Variable	loan_status - either 'Approved' or 'Rejected'
Source Format	CSV (Comma-Separated Values)
Missing Values	None -the dataset is complete with no null entries

The 13 columns in the dataset are:

loan_id - Unique identifier for each application (not used in modelling)

no_of_dependents -Number of people financially dependent on the applicant

education - Whether the applicant is a 'Graduate' or 'Not Graduate'

self_employed - Whether the applicant runs their own business ('Yes' or 'No')

income_annum - Annual income of the applicant (in Indian Rupees)

loan_amount - The total loan amount requested

loan_term -Duration of the loan (in months)

cibil_score -Credit score ranging from 300 (poor) to 900 (excellent)

residential_assets_value - Market value of home/residential property

commercial_assets_value - Market value of commercial property

luxury_assets_value -Value of luxury assets.

bank_asset_value -Balance/assets held in bank accounts

loan_status - The outcome: 'Approved' or 'Rejected'

Step 1: Data Collection

Data collection is the very first and most critical step in any machine learning project. Without good data, even the best algorithms will fail to produce reliable results. Think of it as the foundation of a building if the foundation is weak, the whole structure will collapse.

1. The Dataset: We used a file named loan_data.csv. It contains 13 different types of information (features) for each person, such as their Gender, Education, and Property Area.

2. Location: This dataset is already uploaded and available in the GitHub repository within the data/ folder.

3. Action: We used the pandas library to load this data so we could start our analysis.

Initial Data Inspection

After loading the data, we performed an initial inspection to understand its structure:

Inspection Check	Finding
Total rows (applicants)	4,269
Total columns (features)	13
Missing values	None -all 4,269 rows had complete data
Numeric columns	loan_id, no_of_dependents, income_annum, loan_amount, loan_term, cibil_score, and all asset columns
Text (categorical) columns	education, self_employed, loan_status

Approved applications	2,656 (62.2%)
Rejected applications	1,613 (37.8%)

Step 2: Data Preprocessing

Raw data is often messy. Some people might have left parts of their application blank. A computer cannot process "empty" information.

1. Handling Missing Values: We scanned the entire dataset for null (empty) values.

2. The Solution: For columns with numbers (like Loan Amount), we filled the gaps with the Average (Mean). For columns with words (like Credit History), we filled gaps with the Most Frequent Answer (Mode). This ensures our model has a full picture to learn from.

Step 3: Feature Engineering

Machine Learning models are basically big calculators-they only understand numbers, not English words.

1. Label Encoding: We used a technique called "Label Encoding" to translate words into numbers.

Python's LabelEncoder from the scikit-learn library was used to convert these to numbers automatically:

Original Value	Encoded Value
No	0
Yes	1

The education column had two possible values: 'Graduate' and 'Not Graduate'. A special technique called One-Hot Encoding was applied using pd.get_dummies():

Original Value	education_Graduate	education_Not Graduate
Graduate	1	0
Not Graduate	0	1

2. Example: We changed "Graduate" to 1 and "Not Graduate" to 0. We changed "Male" to 1 and "Female" to 0.

3. Result: After this step, the entire dataset (which is updated in notebook) became a table of numbers that our AI "brain" could finally understand.

Step 4: Model Selection

We needed to choose the best algorithm for this job. After researching, we chose the Random Forest Classifier.

1. Why Random Forest? A single "Decision Tree" can sometimes make a mistake. A "Random Forest" is much smarter because it creates 100 different trees at once.

(i) Best for: Complex, real-world datasets with many features.

(ii) Advantage: Highly resistant to overfitting because the trees collectively correct each other's errors.

2. Logic: Each tree gives an opinion, and the model takes the "Majority Vote." This makes our predictions very stable and highly accurate.

Model Comparison

Model	Initial Accuracy	Complexity	Overfitting Risk
Logistic Regression	79.7%	Low	Low
Decision Tree	96.9%	Medium	High
Random Forest	97.7%	High	Low

Step 5: Model Training

Splitting the Data

Before training, the dataset was split into two parts:

Split	Size	Purpose
Training Set	80% (~3,415 records)	Used to train the model - it sees this data during learning
Testing Set	20% (~837 records)	Kept hidden during training; used to evaluate the model's real-world performance

Configuring the Random Forest

The Random Forest model was configured with the following hyperparameters:

Hyperparameter	Value Set	What it Means
n_estimators	200	Number of trees to build in the forest. More trees = more robust.
max_depth	10	Maximum depth of each tree. Limits complexity to prevent overfitting.
random_state	39	A seed value to ensure results are reproducible - same result every time the code runs.

Training Process

Training was performed by calling rf_model.fit(X_train, y_train).

Step 6: Model Evaluation

Model evaluation is the process of measuring how well our trained model performs on data it has never seen before the test set. To make sure our model actually learned and didn't just memorize the answers, we had to give it a test.

1. The Test: We gave the model the remaining 20% of the data that it had never seen before.

2. Action: We calculated the Accuracy **Score**. By comparing the model's guesses against the real results, we proved that the system is reliable enough to be used in a real bank.

Step 7: Model Deployment

Saving the Model

The trained Random Forest model was saved to disk using Python's joblib library:

Deployment Step	Details
Library Used	joblib - part of Python's scientific ecosystem, efficient for large arrays
Output File	loan_approval_prediction.pkl (.pkl = Python Pickle format)
File Purpose	The .pkl file contains the fully trained model - all 200 decision trees with their learned rules
Reloading	Any Python environment can load this file with joblib.load() and immediately use it to make predictions without retraining

Live Prediction Demonstration

To prove the model works in practice, a prediction was run on a completely new, hypothetical applicant who was not in the training data:

Feature	Value for Test Applicant
No. of Dependents	4
Education	Graduate
Self Employed	Yes
Annual Income	₹6,00,000
Loan Amount Requested	₹25,00,000
Loan Term	36 months
CIBIL Score	900 (Maximum possible - Excellent)
Residential Assets	₹3,00,000

Commercial Assets	**₹0**
Luxury Assets	**₹0**
Bank Asset Value	**₹5,000**

Model Prediction Output

LOAN STATUS: APPROVED

Conclusion

By following these 7 detailed steps, we successfully turned a raw CSV file into a smart prediction tool. This project proves that Machine Learning can make complicated banking tasks simple and accurate. All the code, the final model, and the dataset stored in the GitHub repository for future use.

REFRENCES

Category	Tool / Library
Language	Python 3
Data Manipulation	Pandas
Numerical Computing	NumPy
Visualization	Matplotlib + Seaborn
Machine Learning	Scikit-learn
Model Saving	Joblib
Development Environment	Google Colab